```
import os
import pandas as pd

# Directory where your stock files are located
base_dir = r"C:\aFiles And Media\DataAnalytics\HexaSolutions\Hexa
Assignments\Assignments Till 09-10-2026\Assignment FINAL - PROJECTS -
20260510T122027Z-3-001\Assignment FINAL - PROJECTS\7- SQL Stock
Marketing Assignment\SQL - Stock Market Analysis Project"

# Mapping raw CSV filenames to stock names
stock_files = {
    'Bajaj Auto.csv': 'Bajaj Auto',
    'Eicher Motors.csv': 'Eicher Motors',
    'Hero Motocorp.csv': 'Hero Motocorp',
    'Infosys.csv': 'Infosys',
    'TCS.csv': 'TCS',
    'TVS Motors.csv': 'TVS Motors'
}

clean_frames = []

for file_name, stock_name in stock_files.items():
    file_path = os.path.join(base_dir, file_name)

    if not os.path.exists(file_path):
        print(f"Skipping {file_name}: file not found at {file_path}")
        continue

    df = pd.read_csv(file_path)

    # 1. Convert Date to standard SQL format (YYYY-MM-DD)
    df['Date'] = pd.to_datetime(df['Date'], format='%d-%B-
%Y').dt.strftime('%Y-%m-%d')

    # 2. Add Stock_Name column
    df['Stock_Name'] = stock_name

    # 3. Rename Close Price to Closing_Price
    df = df.rename(columns={'Close Price': 'Closing_Price'})

    # 4. Filter only required columns specified in Part 2
    df_clean = df[['Stock_Name', 'Date', 'Closing_Price']].copy()

    # 5. Sort chronologically from earliest to latest
    df_clean = df_clean.sort_values('Date').reset_index(drop=True)

    # Correct file saving inside the target directory
    clean_path = os.path.join(base_dir, f"clean_{file_name}")
    df_clean.to_csv(clean_path, index=False)
    print(f"Saved: {clean_path} ({len(df_clean)} rows)")
```

```
    clean_frames.append(df_clean)

# Merge all into one consolidated master CSV
master_df = pd.concat(clean_frames, ignore_index=True)
master_path = os.path.join(base_dir, "master_stock_prices.csv")
master_df.to_csv(master_path, index=False)

print("\n--- Summary ---")
print(f"Master file saved: {master_path}")
print(f"Total rows: {len(master_df)} (889 rows * 6 stocks)")
print(f"Total null values: {master_df.isnull().sum().sum()}")
```

```
Saved: C:\aFiles And Media\DataAnalytics\HexaSolutions\Hexa
Assignments\Assignments Till 09-10-2026\Assignment FINAL - PROJECTS -
20260510T122027Z-3-001\Assignment FINAL - PROJECTS\7- SQL Stock
Marketing Assignment\SQL - Stock Market Analysis Project\clean_Bajaj
Auto.csv (889 rows)
Saved: C:\aFiles And Media\DataAnalytics\HexaSolutions\Hexa
Assignments\Assignments Till 09-10-2026\Assignment FINAL - PROJECTS -
20260510T122027Z-3-001\Assignment FINAL - PROJECTS\7- SQL Stock
Marketing Assignment\SQL - Stock Market Analysis Project\clean_Eicher
Motors.csv (889 rows)
Saved: C:\aFiles And Media\DataAnalytics\HexaSolutions\Hexa
Assignments\Assignments Till 09-10-2026\Assignment FINAL - PROJECTS -
20260510T122027Z-3-001\Assignment FINAL - PROJECTS\7- SQL Stock
Marketing Assignment\SQL - Stock Market Analysis Project\clean_Hero
Motocorp.csv (889 rows)
Saved: C:\aFiles And Media\DataAnalytics\HexaSolutions\Hexa
Assignments\Assignments Till 09-10-2026\Assignment FINAL - PROJECTS -
20260510T122027Z-3-001\Assignment FINAL - PROJECTS\7- SQL Stock
Marketing Assignment\SQL - Stock Market Analysis Project\
clean_Infosys.csv (889 rows)
Saved: C:\aFiles And Media\DataAnalytics\HexaSolutions\Hexa
Assignments\Assignments Till 09-10-2026\Assignment FINAL - PROJECTS -
20260510T122027Z-3-001\Assignment FINAL - PROJECTS\7- SQL Stock
Marketing Assignment\SQL - Stock Market Analysis Project\clean_TCS.csv
(889 rows)
Saved: C:\aFiles And Media\DataAnalytics\HexaSolutions\Hexa
Assignments\Assignments Till 09-10-2026\Assignment FINAL - PROJECTS -
20260510T122027Z-3-001\Assignment FINAL - PROJECTS\7- SQL Stock
Marketing Assignment\SQL - Stock Market Analysis Project\clean_TVS
Motors.csv (889 rows)

--- Summary ---
Master file saved: C:\aFiles And Media\DataAnalytics\HexaSolutions\
Hexa Assignments\Assignments Till 09-10-2026\Assignment FINAL -
PROJECTS -20260510T122027Z-3-001\Assignment FINAL - PROJECTS\7- SQL
Stock Marketing Assignment\SQL - Stock Market Analysis Project\
master_stock_prices.csv
```

```
Total rows: 5334 (889 rows * 6 stocks)
Total null values: 0
```

```sql
-- 1. Update text into standard ISO format
UPDATE stock_prices
SET Date = STR_TO_DATE(Date, '%d-%m-%Y');

-- 2. Modify column type to DATE
ALTER TABLE master_stock_prices
MODIFY COLUMN Date DATE;

DESCRIBE master_stock_prices;
```

```sql
-- 2. Modify column type to DATE
ALTER TABLE master_stock_prices
MODIFY COLUMN Date DATE;

DESCRIBE master_stock_prices;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

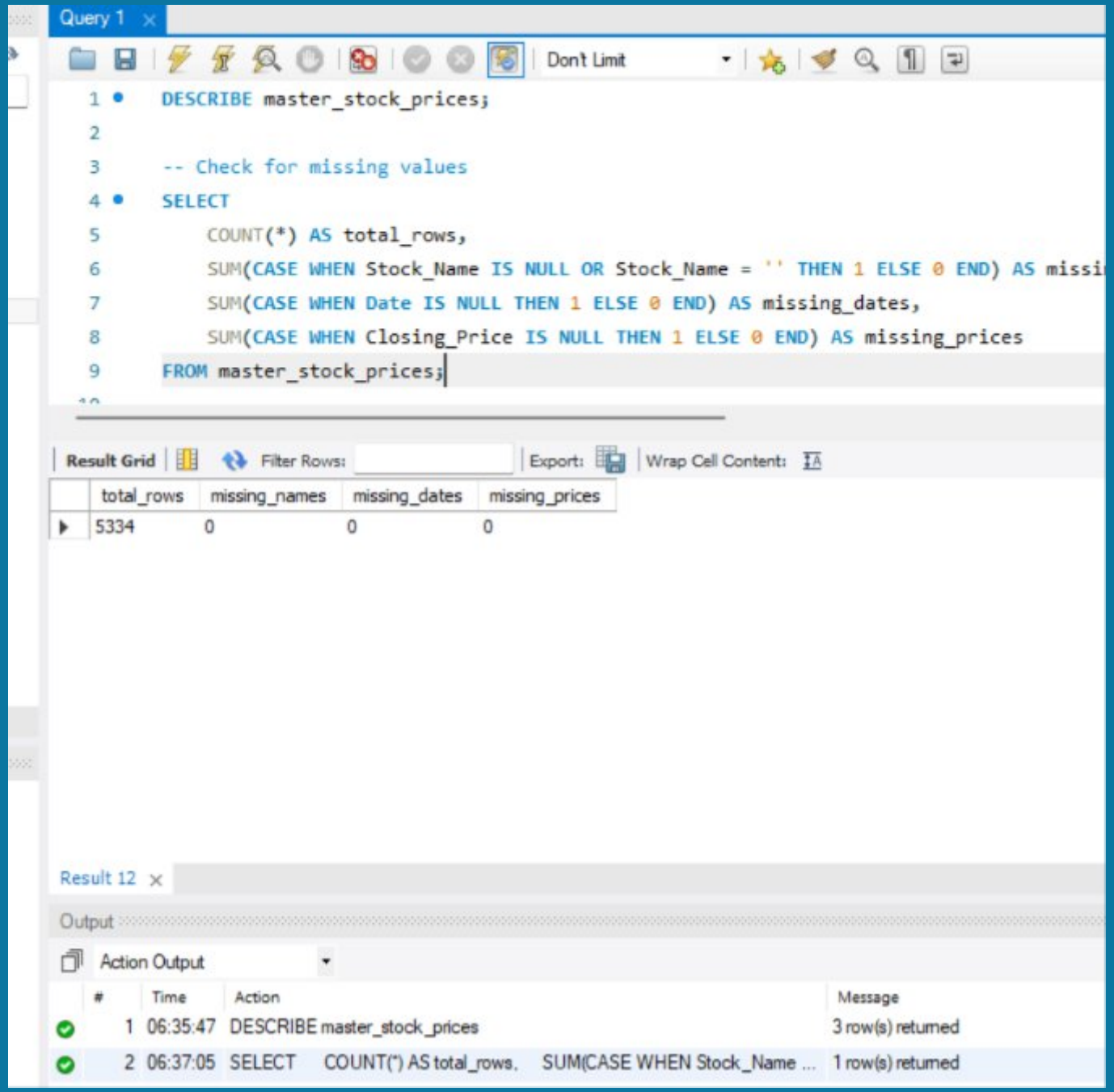
Field	Type	Null	Key	Default	Extra
Stock_Name	text	YES		NULL	
Date	date	YES		NULL	
Closing_Price	double	YES		NULL	

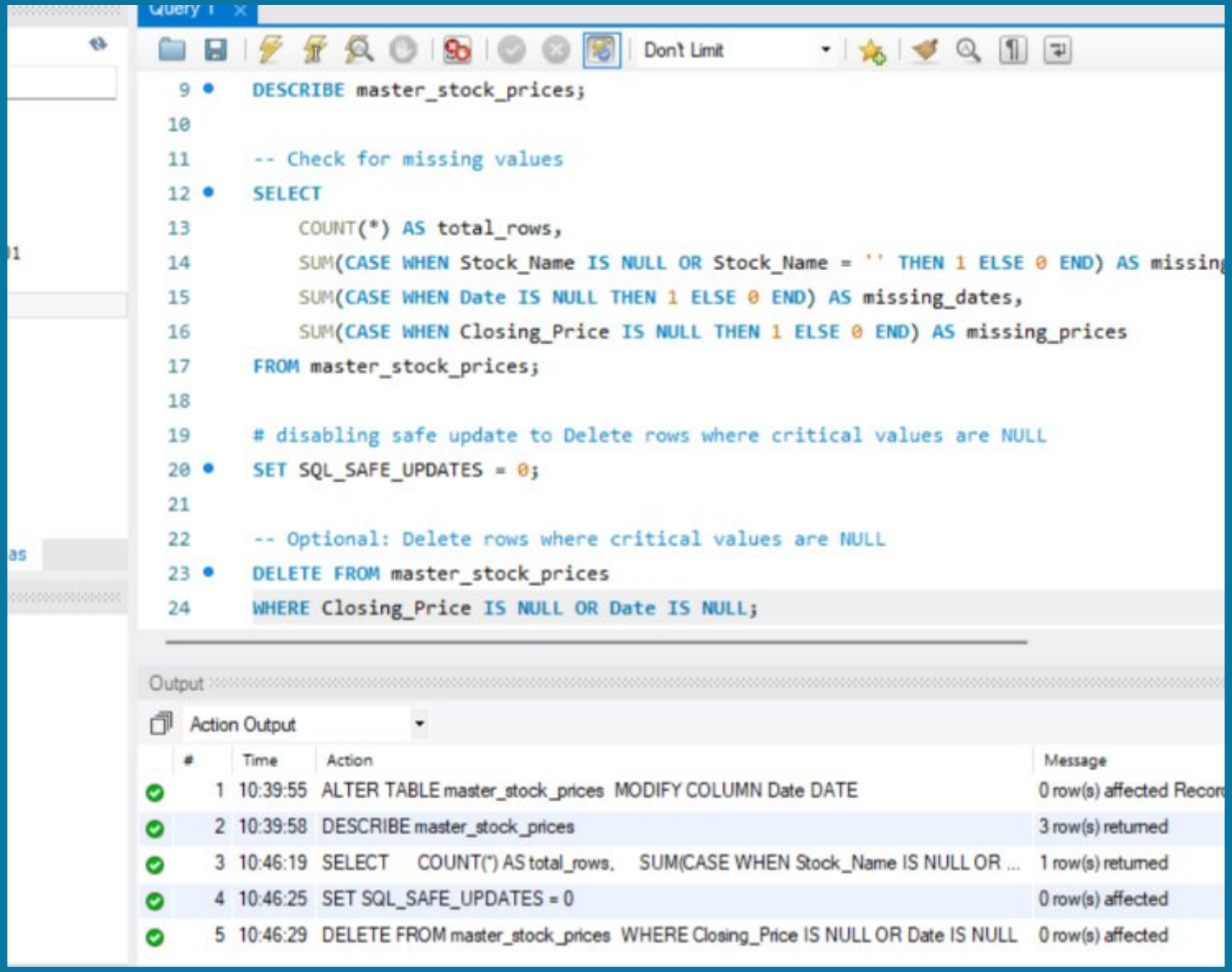
Result 4

Output

Action Output

#	Time	Action	Message
1	10:39:55	ALTER TABLE master_stock_prices MODIFY COLUMN Date DATE	0 row(s) affected R
2	10:39:58	DESCRIBE master_stock_prices	3 row(s) returned

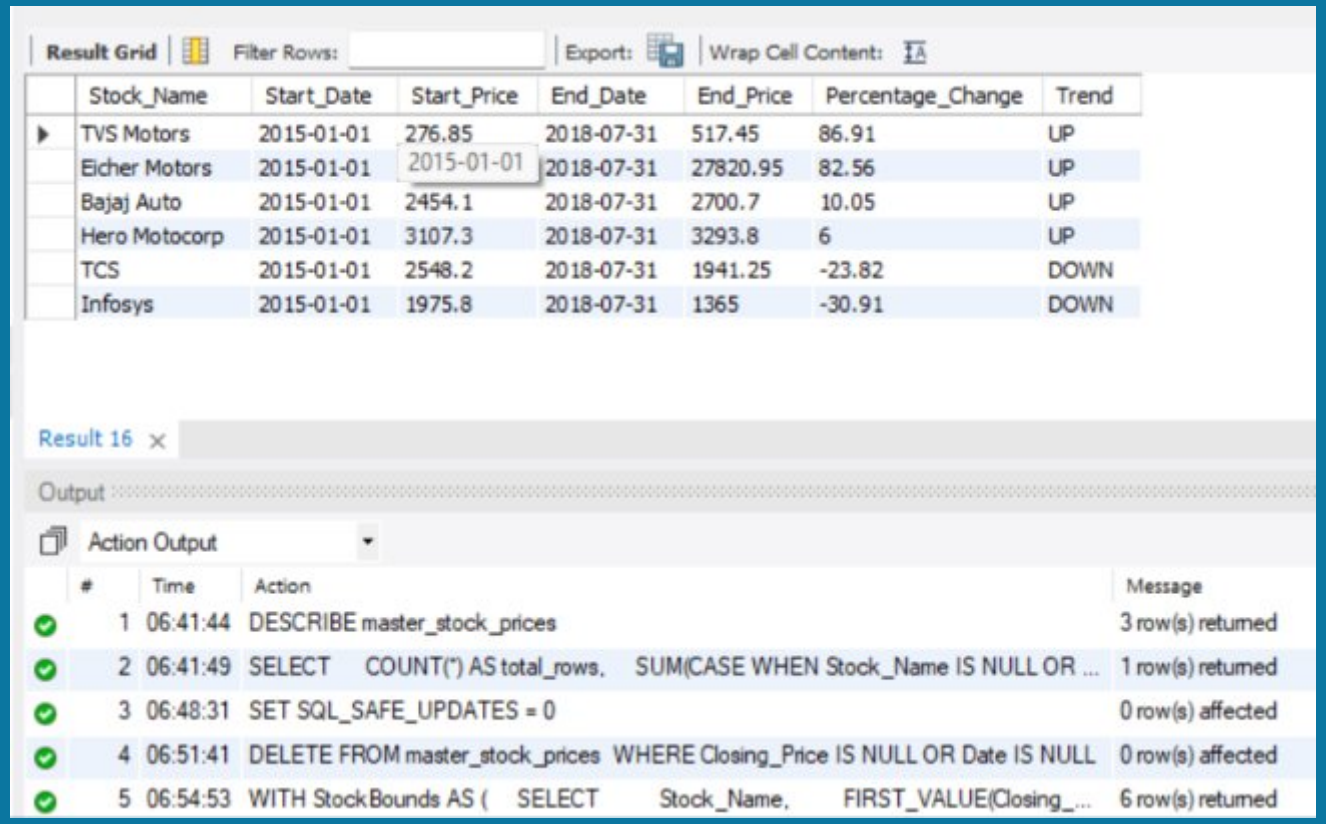

Query 1
Don't Limit
1 DESCRIBE master_stock_prices;
2
3 -- Check for missing values
4 SELECT
5 COUNT(*) AS total_rows,
6 SUM(CASE WHEN Stock_Name IS NULL OR Stock_Name = '' THEN 1 ELSE 0 END) AS missi
7 SUM(CASE WHEN Date IS NULL THEN 1 ELSE 0 END) AS missing_dates,
8 SUM(CASE WHEN Closing_Price IS NULL THEN 1 ELSE 0 END) AS missing_prices
9 FROM master_stock_prices;
Result Grid
Filter Rows:
Export:
Wrap Cell Content:
total_rows missing_names missing_dates missing_prices
5334 0 0 0
Result 12
Output
Action Output
Time Action Message
1 06:35:47 DESCRIBE master_stock_prices 3 row(s) returned
2 06:37:05 SELECT COUNT(*) AS total_rows, SUM(CASE WHEN Stock_Name ... 1 row(s) returned


Query 1
Don't Limit
9 DESCRIBE master_stock_prices;
10
11 -- Check for missing values
12 SELECT
13 COUNT(*) AS total_rows,
14 SUM(CASE WHEN Stock_Name IS NULL OR Stock_Name = '' THEN 1 ELSE 0 END) AS missing
15 SUM(CASE WHEN Date IS NULL THEN 1 ELSE 0 END) AS missing_dates,
16 SUM(CASE WHEN Closing_Price IS NULL THEN 1 ELSE 0 END) AS missing_prices
17 FROM master_stock_prices;
18
19 # disabling safe update to Delete rows where critical values are NULL
20 SET SQL_SAFE_UPDATES = 0;
21
22 -- Optional: Delete rows where critical values are NULL
23 DELETE FROM master_stock_prices
24 WHERE Closing_Price IS NULL OR Date IS NULL;
Output
Action Output
Time Action Message
1 10:39:55 ALTER TABLE master_stock_prices MODIFY COLUMN Date DATE 0 row(s) affected Recor
2 10:39:58 DESCRIBE master_stock_prices 3 row(s) returned
3 10:46:19 SELECT COUNT(*) AS total_rows, SUM(CASE WHEN Stock_Name IS NULL OR ... 1 row(s) returned
4 10:46:25 SET SQL_SAFE_UPDATES = 0 0 row(s) affected
5 10:46:29 DELETE FROM master_stock_prices WHERE Closing_Price IS NULL OR Date IS NULL 0 row(s) affected

```sql
-- Trend Analysis (Start to End Percentage Change)

WITH StockBounds AS (
    SELECT
        Stock_Name,
        FIRST_VALUE(Closing_Price) OVER (
            PARTITION BY Stock_Name ORDER BY Date ASC
        ) AS Start_Price,
        FIRST_VALUE(Closing_Price) OVER (
            PARTITION BY Stock_Name ORDER BY Date DESC
        ) AS End_Price,
        FIRST_VALUE(Date) OVER (
            PARTITION BY Stock_Name ORDER BY Date ASC
        ) AS Start_Date,
        FIRST_VALUE(Date) OVER (
            PARTITION BY Stock_Name ORDER BY Date DESC
        ) AS End_Date,
        ROW_NUMBER() OVER (
            PARTITION BY Stock_Name ORDER BY Date DESC
        ) AS rn
    FROM master_stock_prices
)
```

```sql
SELECT
    Stock_Name,
    Start_Date,
    Start_Price,
    End_Date,
    End_Price,
    ROUND(((End_Price - Start_Price) / Start_Price) * 100, 2) AS Percentage_Change,
    CASE
        WHEN End_Price >= Start_Price THEN 'UP'
        ELSE 'DOWN'
    END AS Trend
FROM StockBounds
WHERE rn = 1
ORDER BY Percentage_Change DESC;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Stock_Name	Start_Date	Start_Price	End_Date	End_Price	Percentage_Change	Trend
TVS Motors	2015-01-01	276.85	2018-07-31	517.45	86.91	UP
Eicher Motors	2015-01-01	2015-01-01	2018-07-31	27820.95	82.56	UP
Bajaj Auto	2015-01-01	2454.1	2018-07-31	2700.7	10.05	UP
Hero Motocorp	2015-01-01	3107.3	2018-07-31	3293.8	6	UP
TCS	2015-01-01	2548.2	2018-07-31	1941.25	-23.82	DOWN
Infosys	2015-01-01	1975.8	2018-07-31	1365	-30.91	DOWN

Result 16

Output

Action Output

#	Time	Action	Message
1	06:41:44	DESCRIBE master_stock_prices	3 row(s) returned
2	06:41:49	SELECT COUNT(*) AS total_rows, SUM(CASE WHEN Stock_Name IS NULL OR ...	1 row(s) returned
3	06:48:31	SET SQL_SAFE_UPDATES = 0	0 row(s) affected
4	06:51:41	DELETE FROM master_stock_prices WHERE Closing_Price IS NULL OR Date IS NULL	0 row(s) affected
5	06:54:53	WITH StockBounds AS (SELECT Stock_Name, FIRST_VALUE(Closing_...	6 row(s) returned

```sql
-- Buy and Sell Signals

SELECT
    Stock_Name,
    Date,
    Closing_Price,
    ROUND(AVG(Closing_Price) OVER (
        PARTITION BY Stock_Name
        ORDER BY Date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ), 2) AS Historical_Avg_Price,
    CASE
        WHEN Closing_Price > AVG(Closing_Price) OVER (
            PARTITION BY Stock_Name
            ORDER BY Date
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
        ) THEN 'BUY'
        ELSE 'SELL'
    END AS Signal_Recommendation
FROM master_stock_prices
ORDER BY Stock_Name, Date;
```

```
            ) THEN 'BUY'
            ELSE 'SELL'
        END AS Signal_Recommendation
    FROM master_stock_prices
    ORDER BY Stock_Name, Date;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

Stock_Name	Date	Closing_Price	Historical_Avg_Price	Signal_Recommendation
Bajaj Auto	2015-01-01	2454.1	2454.1	SELL
Bajaj Auto	2015-01-02	2453.5	2453.8	SELL
Bajaj Auto	2015-01-05	2460.15	2455.92	BUY
Bajaj Auto	2015-01-06	2440.35	2452.02	SELL
Bajaj Auto	2015-01-07	2447.2	2451.06	SELL
Bajaj Auto	2015-01-08	2450.05	2450.89	SELL
Bajaj Auto	2015-01-09	2381.5	2440.98	SELL
Bajaj Auto	2015-01-12	2333.25	2427.51	SELL
Bajaj Auto	2015-01-13	2326.25	2416.26	SELL

Result 17

Output

Action Output

#	Time	Action	Message
1	06:41:44	DESCRIBE master_stock_prices	3 row(s) returned
2	06:41:49	SELECT COUNT(*) AS total_rows, SUM(CASE WHEN Stock_Name IS NULL OR ...	1 row(s) returned
3	06:48:31	SET SQL_SAFE_UPDATES = 0	0 row(s) affected
4	06:51:41	DELETE FROM master_stock_prices WHERE Closing_Price IS NULL OR Date IS NULL	0 row(s) affected
5	06:54:53	WITH StockBounds AS (SELECT Stock_Name, FIRST_VALUE(Closing_...	6 row(s) returned
6	06:58:46	SELECT Stock_Name, Date, Closing_Price, ROUND(AVG(Closing_Price) OV...	5334 row(s) returned

```sql
-- Maximum and Minimum Stock Price

SELECT
    Stock_Name,
    MIN(Closing_Price) AS Min_Price,
    MAX(Closing_Price) AS Max_Price,
    ROUND(MAX(Closing_Price) - MIN(Closing_Price), 2) AS Price_Spread
FROM master_stock_prices
GROUP BY Stock_Name
ORDER BY Stock_Name;
```

```
81    FROM master_stock_prices
82    GROUP BY Stock_Name
83    ORDER BY Stock_Name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Stock_Name	Min_Price	Max_Price	Price_Spread
Bajaj Auto	1949.4	3409.5	1460.1
Eicher Motors	14320.6	32786.4	18465.8
Hero Motocorp	2279.8	4047.3	1767.5
Infosys	873.5	2326.6	1453.1
TCS	1721.2	3604.8	1883.6
TVS Motors	213.55	785.3	571.75

Result 18

Output

Action Output

#	Time	Action	Message
2	06:41:49	SELECT COUNT(*) AS total_rows, SUM(CASE WHEN Stock_Name IS NULL O...	1 row(s) returned
3	06:48:31	SET SQL_SAFE_UPDATES = 0	0 row(s) affected
4	06:51:41	DELETE FROM master_stock_prices WHERE Closing_Price IS NULL OR Date IS N...	0 row(s) affected
5	06:54:53	WITH StockBounds AS (SELECT Stock_Name, FIRST_VALUE(Closing...	6 row(s) returned
6	06:58:46	SELECT Stock_Name, Date, Closing_Price, ROUND(AVG(Closing_Price) O...	5334 row(s) returned
7	06:59:55	SELECT Stock_Name, MIN(Closing_Price) AS Min_Price, MAX(Closing_Price) ...	6 row(s) returned

```sql
SELECT
    p.Stock_Name,
    COUNT(CASE WHEN p.Closing_Price = bounds.Min_Price THEN 1 END) AS Lowest_Buy_Opportunities,
    COUNT(CASE WHEN p.Closing_Price = bounds.Max_Price THEN 1 END) AS Highest_Sell_Opportunities
FROM master_stock_prices p
JOIN (
    SELECT
        Stock_Name,
        MIN(Closing_Price) AS Min_Price,
        MAX(Closing_Price) AS Max_Price
    FROM master_stock_prices
    GROUP BY Stock_Name
) bounds ON p.Stock_Name = bounds.Stock_Name
GROUP BY p.Stock_Name
ORDER BY p.Stock_Name;
```

```
97      FROM master_stock_prices
98      GROUP BY Stock_Name
99  ) bounds ON p.Stock_Name = bounds.Stock_Name
100 GROUP BY p.Stock_Name
101 ORDER BY p.Stock_Name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Stock_Name	Lowest_Buy_Opportunities	Highest_Sell_Opportunities
Bajaj Auto	1	1
Eicher Motors	1	1
Hero Motocorp	1	1
Infosys	1	1
TCS	1	1
TVS Motors	1	1

esult 20 ×

utput

Action Output

#	Time	Action	Message
5	06:54:53	WITH StockBounds AS (SELECT Stock_Name, FIRST_VALUE(Closing...	6 row(s) returned
6	06:58:46	SELECT Stock_Name, Date, Closing_Price, ROUND(AVG(Closing_Price) O...	5334 row(s) returne
7	06:59:55	SELECT Stock_Name, MIN(Closing_Price) AS Min_Price, MAX(Closing_Price) ...	6 row(s) returned
8	07:02:16	SELECT p.Stock_Name, COUNT(CASE WHEN p.Closing_Price = bounds.Min_P...	Error Code: 1146.
9	07:03:00	SELECT p.Stock_Name, COUNT(CASE WHEN p.Closing_Price = bounds.Min_P...	6 row(s) returned
10	07:04:19	SELECT p.Stock_Name, COUNT(CASE WHEN p.Closing_Price = bounds.Min_P...	6 row(s) returned

```sql
-- Percentage Change Analysis And Ranking

WITH StartEndPrices AS (
    SELECT
        Stock_Name,
        FIRST_VALUE(Closing_Price) OVER (
            PARTITION BY Stock_Name ORDER BY Date ASC
        ) AS Start_Price,
        FIRST_VALUE(Closing_Price) OVER (
            PARTITION BY Stock_Name ORDER BY Date DESC
        ) AS End_Price,
        ROW_NUMBER() OVER (
            PARTITION BY Stock_Name ORDER BY Date DESC
        ) AS rn
    FROM master_stock_prices
)
```

```sql
SELECT
    RANK() OVER (ORDER BY ((End_Price - Start_Price) / Start_Price) * 100 DESC) AS Performance_Rank,
    Stock_Name,
    Start_Price,
    End_Price,
    ROUND(((End_Price - Start_Price) / Start_Price) * 100, 2) AS Percentage_Change
FROM StartEndPrices
WHERE rn = 1;
```

```
    Stock_Name,
    Start_Price,
    End_Price,
    ROUND(((End_Price - Start_Price) / Start_Price) * 100, 2) AS Percentage_Change
FROM StartEndPrices
WHERE rn = 1;
```

lt Grid | Filter Rows: | Export: | Wrap Cell Content:

Performance_Rank	Stock_Name	Start_Price	End_Price	Percentage_Change
	TVS Motors	276.85	517.45	86.91
	Eicher Motors	15239.15	27820.95	82.56
	Bajaj Auto	2454.1	2700.7	10.05
	Hero Motocorp	3107.3	3293.8	6
	TCS	2548.2	1941.25	-23.82
	Infosys	1975.8	1365	-30.91

lt 22 ×

Action Output

	Time	Action	Message
7	06:59:55	SELECT Stock_Name, MIN(Closing_Price) AS Min_Price, MAX(Closing_Price) ...	6 row(s) returned
8	07:02:16	SELECT p.Stock_Name, COUNT(CASE WHEN p.Closing_Price = bounds.Min_P...	Error Code: 1146. Table
9	07:03:00	SELECT p.Stock_Name, COUNT(CASE WHEN p.Closing_Price = bounds.Min_P...	6 row(s) returned
10	07:04:19	SELECT p.Stock_Name, COUNT(CASE WHEN p.Closing_Price = bounds.Min_P...	6 row(s) returned
11	07:06:14	WITH StartEndPrices AS (SELECT Stock_Name, FIRST_VALUE(Closin...	6 row(s) returned
12	07:07:27	WITH StartEndPrices AS (SELECT Stock_Name, FIRST_VALUE(Closin...	6 row(s) returned

```sql

-- Insights on Potential Profit

SELECT
    Stock_Name,
    MIN(Closing_Price) AS Min_Buy_Price,
    MAX(Closing_Price) AS Max_Sell_Price,
    ROUND(MAX(Closing_Price) - MIN(Closing_Price), 2) AS Max_Profit_Per_Share,
    ROUND(((MAX(Closing_Price) - MIN(Closing_Price)) / MIN(Closing_Price)) * 100, 2) AS Max_Potential_Return_Pct
FROM master_stock_prices
GROUP BY Stock_Name
ORDER BY Max_Potential_Return_Pct DESC;
```

```
136     FROM master_stock_prices
137     GROUP BY Stock_Name
138     ORDER BY Max_Potential_Return_Pct DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Stock_Name	Min_Buy_Price	Max_Sell_Price	Max_Profit_Per_Share	Max_Potential_Return_Pct
TVS Motors	213.55	785.3	571.75	267.74
Infosys	873.5	2326.6	1453.1	166.35
Eicher Motors	14320.6	32786.4	18465.8	128.95
TCS	1721.2	3604.8	1883.6	109.44
Hero Motocorp	2279.8	4047.3	1767.5	77.53
Bajaj Auto	1949.4	3409.5	1460.1	74.9

Result 24

Output

Action Output

#	Time	Action	Message
9	07:03:00	SELECT p.Stock_Name, COUNT(CASE WHEN p.Closing_Price = bounds.Min_P...	6 row(s) returned
10	07:04:19	SELECT p.Stock_Name, COUNT(CASE WHEN p.Closing_Price = bounds.Min_P...	6 row(s) returned
11	07:06:14	WITH StartEndPrices AS (SELECT Stock_Name, FIRST_VALUE(Closin...	6 row(s) returned
12	07:07:27	WITH StartEndPrices AS (SELECT Stock_Name, FIRST_VALUE(Closin...	6 row(s) returned
13	07:09:39	SELECT Stock_Name, MIN(Closing_Price) AS Min_Buy_Price, MAX(Closing_...	6 row(s) returned
14	07:10:19	SELECT Stock_Name, MIN(Closing_Price) AS Min_Buy_Price, MAX(Closing_...	6 row(s) returned

```
!pip install streamlit mysql-connector-python pandas plotly sqlalchemy

Requirement already satisfied: streamlit in C:\Users\prajw\AppData\
Local\Programs\Python\Python314\Lib\site-packages (1.64.0)
Requirement already satisfied: mysql-connector-python in C:\Users\
prajw\AppData\Local\Programs\Python\Python314\Lib\site-packages
(26.7.0)
Requirement already satisfied: pandas in C:\Users\prajw\AppData\Local\
Programs\Python\Python314\Lib\site-packages (3.0.5)
Requirement already satisfied: plotly in C:\Users\prajw\AppData\Local\
Programs\Python\Python314\Lib\site-packages (7.1.0)
Requirement already satisfied: sqlalchemy in C:\Users\prajw\AppData\
Local\Programs\Python\Python314\Lib\site-packages (2.1.3)
Requirement already satisfied: altair!=5.4.0,!=5.4.1,<7,>=5.0.0 in C:\
Users\prajw\AppData\Local\Programs\Python\Python314\Lib\site-packages
(from streamlit) (6.3.0)
Requirement already satisfied: click<9,>=7.0 in C:\Users\prajw\
AppData\Local\Programs\Python\Python314\Lib\site-packages (from
streamlit) (8.5.0)
Requirement already satisfied: numpy<3,>=1.23 in C:\Users\prajw\
AppData\Local\Programs\Python\Python314\Lib\site-packages (from
streamlit) (2.5.1)
Requirement already satisfied: packaging>=20 in C:\Users\prajw\
AppData\Local\Programs\Python\Python314\Lib\site-packages (from
streamlit) (26.2)
Requirement already satisfied: pillow<13,>=7.1.0 in C:\Users\prajw\
AppData\Local\Programs\Python\Python314\Lib\site-packages (from
streamlit) (12.3.0)
Requirement already satisfied: pydeck<1,>=0.8.0b4 in C:\Users\prajw\
AppData\Local\Programs\Python\Python314\Lib\site-packages (from
streamlit) (0.9.3)
Requirement already satisfied: protobuf<8,>=5.26.1 in C:\Users\prajw\
AppData\Local\Programs\Python\Python314\Lib\site-packages (from
streamlit) (7.36.2)
Requirement already satisfied: pyarrow!=25.0.0,<26,>=7.0 in C:\Users\
prajw\AppData\Local\Programs\Python\Python314\Lib\site-packages (from
streamlit) (25.0.1)
Requirement already satisfied: requests<3,>=2.27 in C:\Users\prajw\
AppData\Local\Programs\Python\Python314\Lib\site-packages (from
streamlit) (2.34.2)
Requirement already satisfied: toml<2,>=0.10.1 in C:\Users\prajw\
AppData\Local\Programs\Python\Python314\Lib\site-packages (from
streamlit) (0.10.2)
Requirement already satisfied: typing-extensions<5,>=4.10.0 in C:\
Users\prajw\AppData\Local\Programs\Python\Python314\Lib\site-packages
(from streamlit) (4.16.0)
Requirement already satisfied: starlette<2,>=0.46.0 in C:\Users\prajw\
AppData\Local\Programs\Python\Python314\Lib\site-packages (from
streamlit) (1.7.0)
Requirement already satisfied: uvicorn<1,>=0.30.0 in C:\Users\prajw\
```

```
AppData\Local\Programs\Python\Python314\Lib\site-packages (from
streamlit) (0.54.0)
Requirement already satisfied: httptools<1,>=0.6.3 in C:\Users\prajw\
AppData\Local\Programs\Python\Python314\Lib\site-packages (from
streamlit) (0.8.0)
Requirement already satisfied: anyio<5,>=4.0.0 in C:\Users\prajw\
AppData\Local\Programs\Python\Python314\Lib\site-packages (from
streamlit) (4.15.1)
Requirement already satisfied: python-multipart<1,>=0.0.10 in C:\
Users\prajw\AppData\Local\Programs\Python\Python314\Lib\site-packages
(from streamlit) (0.0.32)
Requirement already satisfied: websockets<17,>=12.0.0 in C:\Users\
prajw\AppData\Local\Programs\Python\Python314\Lib\site-packages (from
streamlit) (16.1.1)
Requirement already satisfied: itsdangerous<3,>=2.1.2 in C:\Users\
prajw\AppData\Local\Programs\Python\Python314\Lib\site-packages (from
streamlit) (2.2.0)
Requirement already satisfied: watchdog<7,>=2.1.5 in C:\Users\prajw\
AppData\Local\Programs\Python\Python314\Lib\site-packages (from
streamlit) (6.0.0)
Requirement already satisfied: python-dateutil>=2.8.2 in C:\Users\
prajw\AppData\Local\Programs\Python\Python314\Lib\site-packages (from
pandas) (2.9.0.post0)
Requirement already satisfied: tzdata in C:\Users\prajw\AppData\Local\
Programs\Python\Python314\Lib\site-packages (from pandas) (2026.3)
Requirement already satisfied: jinja2 in C:\Users\prajw\AppData\Local\
Programs\Python\Python314\Lib\site-packages (from altair!=5.4.0,!
=5.4.1,<7,>=5.0.0->streamlit) (3.1.6)
Requirement already satisfied: jsonschema>=3.0 in C:\Users\prajw\
AppData\Local\Programs\Python\Python314\Lib\site-packages (from
altair!=5.4.0,!=5.4.1,<7,>=5.0.0->streamlit) (4.26.0)
Requirement already satisfied: narwhals>=2.4.0 in C:\Users\prajw\
AppData\Local\Programs\Python\Python314\Lib\site-packages (from
altair!=5.4.0,!=5.4.1,<7,>=5.0.0->streamlit) (2.24.0)
Requirement already satisfied: idna>=2.8 in C:\Users\prajw\AppData\
Local\Programs\Python\Python314\Lib\site-packages (from
anyio<5,>=4.0.0->streamlit) (3.18)
Requirement already satisfied: charset_normalizer<4,>=2 in C:\Users\
prajw\AppData\Local\Programs\Python\Python314\Lib\site-packages (from
requests<3,>=2.27->streamlit) (3.4.9)
Requirement already satisfied: urllib3<3,>=1.26 in C:\Users\prajw\
AppData\Local\Programs\Python\Python314\Lib\site-packages (from
requests<3,>=2.27->streamlit) (2.7.0)
Requirement already satisfied: certifi>=2023.5.7 in C:\Users\prajw\
AppData\Local\Programs\Python\Python314\Lib\site-packages (from
requests<3,>=2.27->streamlit) (2026.7.22)
Requirement already satisfied: h11>=0.8 in C:\Users\prajw\AppData\
Local\Programs\Python\Python314\Lib\site-packages (from
uvicorn<1,>=0.30.0->streamlit) (0.16.0)
```

```
Requirement already satisfied: MarkupSafe>=2.0 in C:\Users\prajw\
AppData\Local\Programs\Python\Python314\Lib\site-packages (from
jinja2->altair!=5.4.0,!=5.4.1,<7,>=5.0.0->streamlit) (3.0.3)
Requirement already satisfied: attrs>=22.2.0 in C:\Users\prajw\
AppData\Local\Programs\Python\Python314\Lib\site-packages (from
jsonschema>=3.0->altair!=5.4.0,!=5.4.1,<7,>=5.0.0->streamlit) (26.1.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in
C:\Users\prajw\AppData\Local\Programs\Python\Python314\Lib\site-
packages (from jsonschema>=3.0->altair!=5.4.0,!=5.4.1,<7,>=5.0.0-
>streamlit) (2025.9.1)
Requirement already satisfied: referencing>=0.28.4 in C:\Users\prajw\
AppData\Local\Programs\Python\Python314\Lib\site-packages (from
jsonschema>=3.0->altair!=5.4.0,!=5.4.1,<7,>=5.0.0->streamlit) (0.37.0)
Requirement already satisfied: rpds-py>=0.25.0 in C:\Users\prajw\
AppData\Local\Programs\Python\Python314\Lib\site-packages (from
jsonschema>=3.0->altair!=5.4.0,!=5.4.1,<7,>=5.0.0->streamlit)
(2026.6.3)
Requirement already satisfied: six>=1.5 in C:\Users\prajw\AppData\
Local\Programs\Python\Python314\Lib\site-packages (from python-
dateutil>=2.8.2->pandas) (1.17.0)


[notice] A new release of pip is available: 26.1.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```python
from sqlalchemy import create_engine
from sqlalchemy.engine import URL
import pandas as pd

connection_url = URL.create(
    drivername="mysql+mysqlconnector",
    username="root",
    password="Pradi@mysql",
    host="localhost",
    database="sma"
)

engine = create_engine(connection_url)

df = pd.read_sql("SELECT * FROM master_stock_prices", engine)
print(df.head())
print(df.tail())
```

```
   Stock_Name        Date  Closing_Price
0  Bajaj Auto  2015-01-01        2454.10
1  Bajaj Auto  2015-01-02        2453.50
2  Bajaj Auto  2015-01-05        2460.15
3  Bajaj Auto  2015-01-06        2440.35
4  Bajaj Auto  2015-01-07        2447.20
      Stock_Name        Date  Closing_Price
```

```
5329  TVS Motors  2018-07-25         540.50
5330  TVS Motors  2018-07-26         524.85
5331  TVS Motors  2018-07-27         526.10
5332  TVS Motors  2018-07-30         513.65
5333  TVS Motors  2018-07-31         517.45
```

```python
import streamlit as st
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from sqlalchemy import create_engine

# --- Page Configuration ---
st.set_page_config(
    page_title="Stock Market Analytics Dashboard",
    layout="wide"
)

# --- Title and Header ---
st.title("Stock Market Analysis And Signal Dashboard")
st.markdown("Automated technical trend analysis, buy/sell signal detection, and return comparison.")

# --- Step A: Data Loading (MySQL with CSV Fallback) ---
@st.cache_data
def load_data():
    try:
        # Replace with your MySQL credentials
        engine = create_engine("mysql+mysqlconnector://root:password@localhost/stock_db")
        query = "SELECT Stock_Name, Date, Closing_Price FROM stock_prices ORDER BY Stock_Name, Date"
        data = pd.read_sql(query, engine)
        st.sidebar.success(" Connected to MySQL database")
    except Exception as e:
        # Fallback to local CSV if MySQL is unreachable
        data = pd.read_csv(r'C:\aFiles And Media\DataAnalytics\HexaSolutions\Hexa Assignments\Assignments To Submit 09-10-2026\SQL Stock Marketing Assignment\7- SQL Stock Marketing Assignment\SQL - Stock Market Analysis Project\master_stock_prices.csv')
        st.sidebar.info(" Loaded from local CSV file (MySQL offline)")

    data['Date'] = pd.to_datetime(data['Date'])
    data.sort_values(by=['Stock_Name', 'Date'], inplace=True)
    return data

df = load_data()

# --- Step B: Sidebar Controls ---
st.sidebar.header("Filter Options")
stock_list = sorted(df['Stock_Name'].unique().tolist())
selected_stock = st.sidebar.selectbox("Select a Company", stock_list, index=0)

min_date = df['Date'].min().date()
max_date = df['Date'].max().date()
date_range = st.sidebar.date_input("Select Date Range", [min_date, max_date], min_value=min_date, max_value=max_date)

# Filter by selected date range
if len(date_range) == 2:
    start_filter, end_filter = date_range
    filtered_df = df[(df['Date'].dt.date >= start_filter) & (df['Date'].dt.date <= end_filter)]
else:
    filtered_df = df.copy()

# Focus dataframe for the selected stock
stock_df = filtered_df[filtered_df['Stock_Name'] == selected_stock].copy()

# --- Step C: Compute Metrics and Buy/Sell Signals ---
if not stock_df.empty:
    # 1. Running Average and Signals
    stock_df['Running_Avg'] = stock_df['Closing_Price'].expanding().mean()
    stock_df['Signal'] = stock_df.apply(
        lambda r: 'BUY' if r['Closing_Price'] > r['Running_Avg'] else 'SELL', axis=1
    )

    start_price = stock_df.iloc[0]['Closing_Price']
    end_price = stock_df.iloc[-1]['Closing_Price']
    min_price = stock_df['Closing_Price'].min()
    max_price = stock_df['Closing_Price'].max()
```

```
    overall_change = ((end_price - start_price) / start_price) * 100
    potential_profit = ((max_price - min_price) / min_price) * 100

    # --- Step D: Display Top Metric Cards ---
    col1, col2, col3, col4 = st.columns(4)
    col1.metric("Current / Latest Price", f"₹{end_price:,.2f}", f"{overall_change:.2f}%")
    col2.metric("Min Closing Price", f"₹{min_price:,.2f}")
    col3.metric("Max Closing Price", f"₹{max_price:,.2f}")
    col4.metric("Max Profit Potential", f"{potential_profit:.2f}%")

    st.markdown("---")

    # --- Step E: Interactive Stock Chart with Buy/Sell Signals ---
    st.subheader(f"Price Trend & Dynamic Buy/Sell Signals: {selected_stock}")

    fig = go.Figure()

    # Price Line
    fig.add_trace(go.Scatter(
        x=stock_df['Date'],
        y=stock_df['Closing_Price'],
        mode='lines',
        name='Closing Price',
        line=dict(color='blue', width=2)
    ))

    # Historical Running Average Line
    fig.add_trace(go.Scatter(
        x=stock_df['Date'],
        y=stock_df['Running_Avg'],
        mode='lines',
        name='Running Avg Baseline',
        line=dict(color='yellow', width=1.5, dash='dash')
    ))

    # Buy Signals (Green points)
    buys = stock_df[stock_df['Signal'] == 'BUY']
    fig.add_trace(go.Scatter(
        x=buys['Date'],
        y=buys['Closing_Price'],
        mode='markers',
        name='Buy Signal (Above Avg)',
        marker=dict(symbol='triangle-up', size=5, color='#10B981')
    ))

    # Sell Signals (Red points)
    sells = stock_df[stock_df['Signal'] == 'SELL']
    fig.add_trace(go.Scatter(
        x=sells['Date'],
        y=sells['Closing_Price'],
        mode='markers',
        name='Sell Signal (Below Avg)',
        marker=dict(symbol='triangle-down', size=5, color='#EF4444')
    ))

    fig.update_layout(
        template="plotly_white",
        hovermode="x unified",
        margin=dict(l=20, r=20, t=30, b=20),
        xaxis_title="Date",
        yaxis_title="Price (₹)",
        legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1)
    )
    st.plotly_chart(fig, use_container_width=True)

# --- Step F: Comparative Analysis (All Stocks) ---
st.markdown("---")
st.subheader("Comparative Market Performance (Start vs. End Date)")

# Calculate overall percentage returns across all stocks in selected window
summary_data = []
for name, group in filtered_df.groupby('Stock_Name'):
```

```
    g_sorted = group.sort_values('Date')
    if not g_sorted.empty:
        s_p = g_sorted.iloc[0]['Closing_Price']
        e_p = g_sorted.iloc[-1]['Closing_Price']
        pct = ((e_p - s_p) / s_p) * 100
        summary_data.append({'Stock Name': name, 'Start Price (₹)': s_p, 'End Price (₹)': e_p,
'Percentage Change (%)': round(pct, 2)})

summary_df = pd.DataFrame(summary_data).sort_values(by='Percentage Change (%)',
ascending=False)

comp_col1, comp_col2 = st.columns([1, 1])

with comp_col1:
    bar_fig = px.bar(
        summary_df,
        x='Stock Name',
        y='Percentage Change (%)',
        color='Percentage Change (%)',
        color_continuous_scale='Tealgrn',
        text='Percentage Change (%)',
        title="Overall Return Ranking (%)"
    )
    bar_fig.update_traces(texttemplate='%{text:.2f}%', textposition='outside')
    bar_fig.update_layout(template="plotly_white")
    st.plotly_chart(bar_fig, use_container_width=True)

with comp_col2:
    st.markdown("### Performance Summary Table")
    st.dataframe(summary_df.reset_index(drop=True), use_container_width=True)
```

```
C:\aFiles And Media\DataAnalytics\HexaSolutions\Hexa Assignments\Assignments To Submit 09-10-2026\SQL Stock Marketing As
signment>streamlit run "stock_market_analysis_dashboard.py"
2026-10-06 08:53:38.357 Uvicorn server started on :::8501

  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://10.223.55.149:8501

  Help agents write better Streamlit apps?
  Install the official Streamlit skills by running streamlit skills in your terminal.
```

Loaded from local CSV file (MySQL offline)

Filter Options

Select a Company

Bajaj Auto

Select Date Range

2015/01/01 – 2018/07/31

Stock Market Analysis And Signal Dashboard

Automated technical trend analysis, buy/sell signal detection, and return comparison.

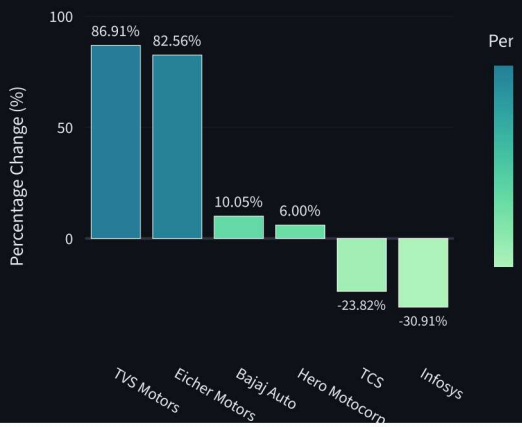
Current / Latest Price	Min Closing Price	Max Closing Price	Max Profit Potential
₹2,700.70	₹1,949.40	₹3,409.50	74.90%
↑ 10.05%			

Price Trend & Dynamic Buy/Sell Signals: Bajaj Auto



Comparative Market Performance (Start vs. End Date)

Overall Return Ranking (%)



Performance Summary Table

	Stock Name	Start Price (₹)	End Price (₹)	Perc
0	TVS Motors	276.85	517.45	
1	Eicher Motors	15239.15	27820.95	
2	Bajaj Auto	2454.1	2700.7	
3	Hero Motocorp	3107.3	3293.8	
4	TCS	2548.2	1941.25	
5	Infosys	1975.8	1365	

Loaded from local CSV file (MySQL offline)

Filter Options

Select a Company

Eicher Motors

Select Date Range

2015/01/01 – 2018/07/31

Stock Market Analysis And Signal Dashboard

Automated technical trend analysis, buy/sell signal detection, and return comparison.

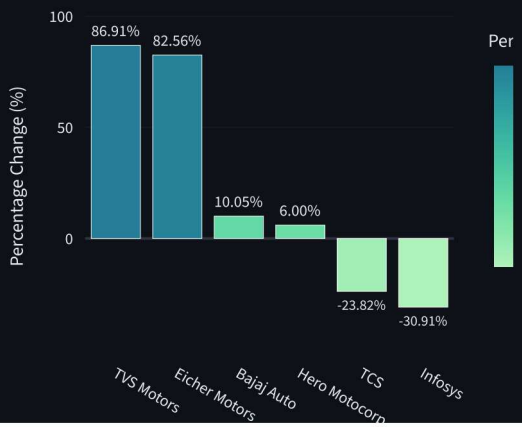
Current / Latest Price	Min Closing Price	Max Closing Price	Max Profit Potential
₹27,820.95	₹14,320.60	₹32,786.40	128.95%
↑ 82.56%			

Price Trend & Dynamic Buy/Sell Signals: Eicher Motors



Comparative Market Performance (Start vs. End Date)

Overall Return Ranking (%)



Performance Summary Table

	Stock Name	Start Price (₹)	End Price (₹)	Perc
0	TVS Motors	276.85	517.45	
1	Eicher Motors	15239.15	27820.95	
2	Bajaj Auto	2454.1	2700.7	
3	Hero Motocorp	3107.3	3293.8	
4	TCS	2548.2	1941.25	
5	Infosys	1975.8	1365	

Loaded from local CSV file (MySQL offline)

Filter Options

Select a Company

Hero Motocorp

Select Date Range

2015/01/01 – 2018/07/31

Stock Market Analysis And Signal Dashboard

Automated technical trend analysis, buy/sell signal detection, and return comparison.

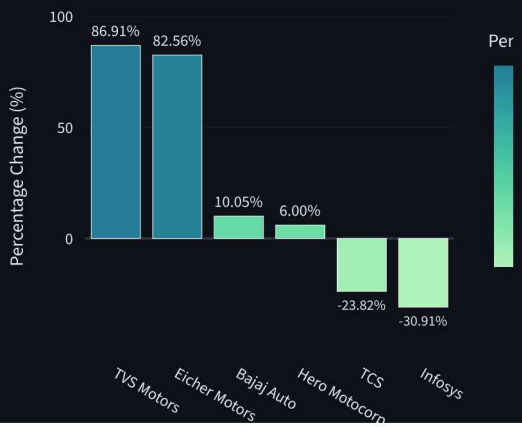
Current / Latest Price	Min Closing Price	Max Closing Price	Max Profit Potential
₹3,293.80	₹2,279.80	₹4,047.30	77.53%
↑ 6.00%			

Price Trend & Dynamic Buy/Sell Signals: Hero Motocorp



Comparative Market Performance (Start vs. End Date)

Overall Return Ranking (%)



Performance Summary Table

	Stock Name	Start Price (₹)	End Price (₹)	Perc
0	TVS Motors	276.85	517.45	
1	Eicher Motors	15239.15	27820.95	
2	Bajaj Auto	2454.1	2700.7	
3	Hero Motocorp	3107.3	3293.8	
4	TCS	2548.2	1941.25	
5	Infosys	1975.8	1365	

Loaded from local CSV file (MySQL offline)

Filter Options

Select a Company

Infosys

Select Date Range

2015/01/01 – 2018/07/31

Stock Market Analysis And Signal Dashboard

Automated technical trend analysis, buy/sell signal detection, and return comparison.

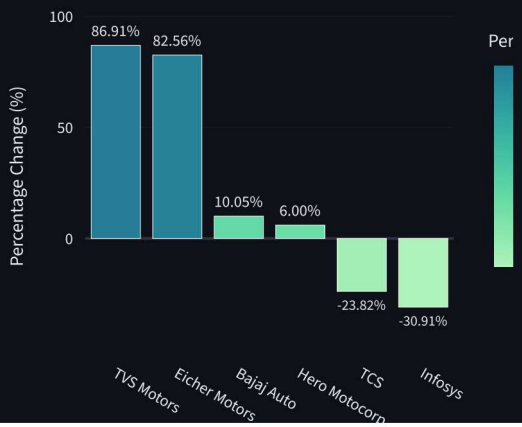
Current / Latest Price	Min Closing Price	Max Closing Price	Max Profit Potential
₹1,365.00	₹873.50	₹2,326.60	166.35%
↓ -30.91%			

Price Trend & Dynamic Buy/Sell Signals: Infosys



Comparative Market Performance (Start vs. End Date)

Overall Return Ranking (%)



Performance Summary Table

	Stock Name	Start Price (₹)	End Price (₹)	Perc
0	TVS Motors	276.85	517.45	
1	Eicher Motors	15239.15	27820.95	
2	Bajaj Auto	2454.1	2700.7	
3	Hero Motocorp	3107.3	3293.8	
4	TCS	2548.2	1941.25	
5	Infosys	1975.8	1365	

Loaded from local CSV file (MySQL offline)

Filter Options

Select a Company

TCS

Select Date Range

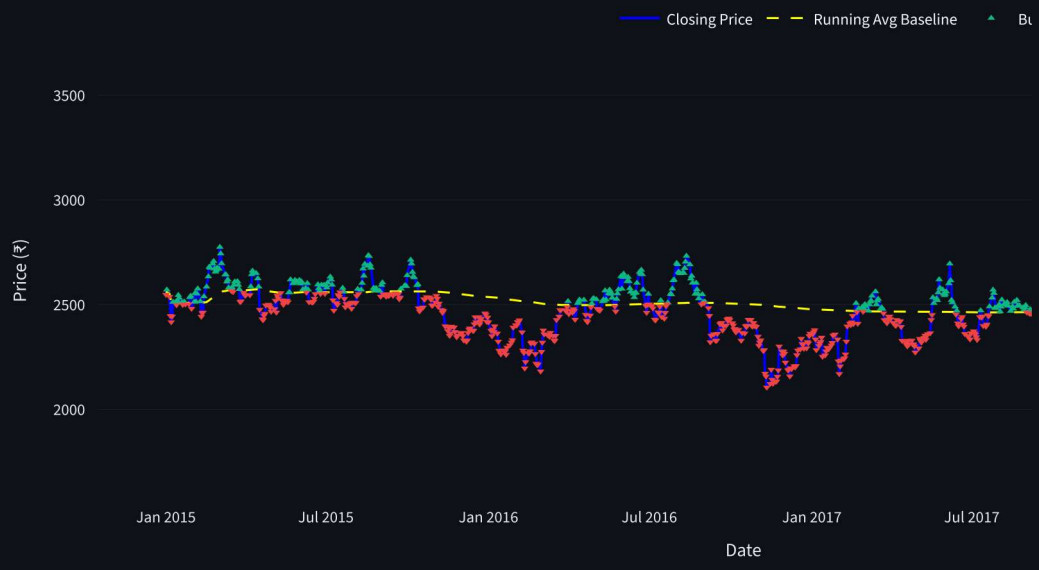
2015/01/01 – 2018/07/31

Stock Market Analysis And Signal Dashboard

Automated technical trend analysis, buy/sell signal detection, and return comparison.

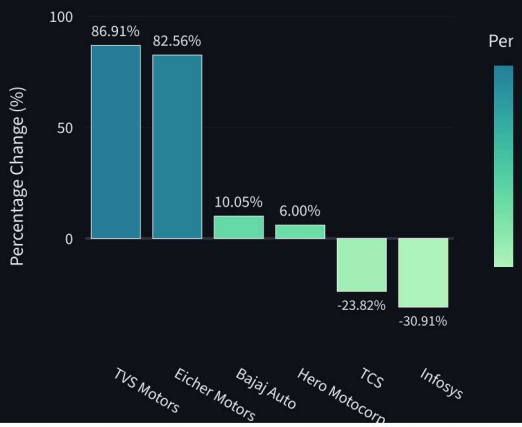
Current / Latest Price	Min Closing Price	Max Closing Price	Max Profit Potential
₹1,941.25	₹1,721.20	₹3,604.80	109.44%
↓ -23.82%			

Price Trend & Dynamic Buy/Sell Signals: TCS



Comparative Market Performance (Start vs. End Date)

Overall Return Ranking (%)



Performance Summary Table

	Stock Name	Start Price (₹)	End Price (₹)	Perc
0	TVS Motors	276.85	517.45	
1	Eicher Motors	15239.15	27820.95	
2	Bajaj Auto	2454.1	2700.7	
3	Hero Motocorp	3107.3	3293.8	
4	TCS	2548.2	1941.25	
5	Infosys	1975.8	1365	

Loaded from local CSV file (MySQL offline)

Filter Options

Select a Company

TVS Motors

Select Date Range

2015/01/01 – 2018/07/31

Stock Market Analysis And Signal Dashboard

Automated technical trend analysis, buy/sell signal detection, and return comparison.

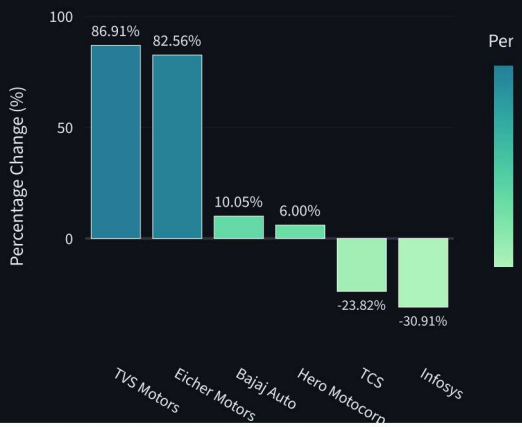
Current / Latest Price	Min Closing Price	Max Closing Price	Max Profit Potential
₹517.45	₹213.55	₹785.30	267.74%
↑ 86.91%			

Price Trend & Dynamic Buy/Sell Signals: TVS Motors



Comparative Market Performance (Start vs. End Date)

Overall Return Ranking (%)



Performance Summary Table

	Stock Name	Start Price (₹)	End Price (₹)	Perc
0	TVS Motors	276.85	517.45	
1	Eicher Motors	15239.15	27820.95	
2	Bajaj Auto	2454.1	2700.7	
3	Hero Motocorp	3107.3	3293.8	
4	TCS	2548.2	1941.25	
5	Infosys	1975.8	1365	

Loaded from local CSV file (MySQL offline)

Filter Options

Select a Company

Bajaj Auto

Select Date Range

2018/06/01 – 2018/07/31

Stock Market Analysis And Signal Dashboard

Automated technical trend analysis, buy/sell signal detection, and return comparison.

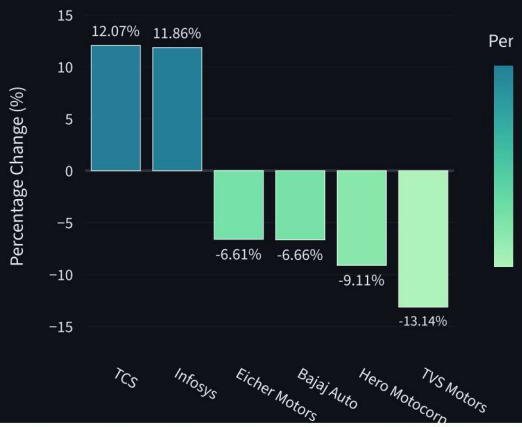
Current / Latest Price	Min Closing Price	Max Closing Price	Max Profit Potential
₹2,700.70	₹2,619.25	₹3,154.35	20.43%
↓ -6.66%			

Price Trend & Dynamic Buy/Sell Signals: Bajaj Auto



Comparative Market Performance (Start vs. End Date)

Overall Return Ranking (%)



Performance Summary Table

	Stock Name	Start Price (₹)	End Price (₹)	Perc
0	TCS	1732.25	1941.25	
1	Infosys	1220.25	1365	
2	Eicher Motors	29788.8	27820.95	
3	Bajaj Auto	2893.25	2700.7	
4	Hero Motocorp	3623.75	3293.8	
5	TVS Motors	595.7	517.45	

Reports and Insights

1. Stock Trends Summary

The analysis of closing prices between January 1, 2015, and July 31, 2018, demonstrates a clear sector divergence between Automobile and Information Technology stocks:

TVS Motors (+86.91% | UP): The best overall performer, growing consistently from ₹276.85 to ₹517.45.

Eicher Motors (+82.56% | UP): Demonstrated sustained long-term compounding, rising from ₹15,239.15 to ₹27,820.95.

Bajaj Auto (+10.05% | UP): Showed moderate net growth, rising from ₹2,454.10 to ₹2,700.70 despite mid-cycle corrections.

Hero Motocorp (+6.00% | UP): Registered steady consolidation with modest overall appreciation from ₹3,107.30 to ₹3,293.80.

TCS (-23.82% | DOWN): Experienced a downward net trajectory across this interval, declining from ₹2,548.20 to ₹1,941.25.

Infosys (-30.91% | DOWN): Faced prolonged correction, falling from ₹1,975.80 to ₹1,365.00

2. Buy/Sell Recommendations

Recommendations are derived from historical price action relative to the expanding average and overall net directional momentum:

Automobile Stocks (Strong Buy / Hold Bias):

TVS Motors & Eicher Motors — Strong Buy: Both stocks maintained sustained trading levels above their historical baseline averages throughout 2016–2018, generating consistent multi-month 'BUY' signal regimes and strong structural upward momentum.

Bajaj Auto & Hero Motocorp — Hold / Accumulate on Dips: These stocks oscillate around their historical averages. Optimal entry signals occur during deep value pullbacks below the dynamic baseline rather than chasing breakout peaks.

IT Stocks (Selective Re-Entry / Trend Sell):

TCS & Infosys — Sell / Defensive Stance on Macro Rallies: Both tech majors spent prolonged periods trading below their dynamic baseline averages, triggering systematic 'SELL' signals throughout 2015–2017. *Short-Term Nuance*: While long-term net returns were negative over the 4-year span, the June–July 2018 dashboard filter shows a sharp late-cycle technical recovery (+12.07% for TCS, +11.86% for Infosys), suggesting cyclical swing-trading opportunities rather than buy-and-hold investing.

3. Insights on Potential Profit (Optimal Low-Buy / High-Sell Strategy)
Evaluating maximum upside potential by buying at the absolute historical minimum and selling at the peak closing high:

Performance Rank	Stock Name	Absolute Low (Buy)	Absolute High (Sell)	Max Spread / Share	Max Potential Profit (%)
1	TVS Motors	₹213.55	₹785.30	₹571.75	267.74%
2	Infosys	₹873.50	₹2,326.60	₹1,453.10	166.35%
3	Eicher Motors	₹14,320.60	₹32,786.40	₹18,465.80	128.95%
4	TCS	₹1,721.20	₹3,604.80	₹1,883.60	109.44%
5	Hero Motocorp	₹2,279.80	₹4,047.30	₹1,767.50	77.53%
6	Bajaj Auto	₹1,949.40	₹3,409.50	₹1,460.10	74.90%

Key finding: Every single stock provided at least a **74% upside opportunity** between its minimum and maximum price cycle.
Even structurally declining stocks like Infosys (+166.35%) and TCS (+109.44%) exhibited substantial swing-trading profit windows, proving that timing cyclical swings can outperform passive holding in range-bound sectors

Final Conclusions :

Sector Winners: The two-wheeler and commercial auto sector (TVS Motors and Eicher Motors) served as the primary growth drivers from 2015 to 2018, rewarding buy-and-hold trend-following investors with returns exceeding **80%**.

Opportunity Scarcity: The SQL buy/sell opportunity query confirms that extreme entry and exit prices occurred on exactly **1 trading day each per stock**, proving that catching the absolute theoretical bottom or top is statistically rare.

Systematic Value: Implementing a dynamic baseline average query effectively captures prolonged trend reversals while filtering out short-term market noise, providing actionable entry and exit signals for portfolio allocation.

Video Link :

https://drive.google.com/file/d/12P9tOkMCD7St6d7sShz5KSSh5l4yINkC/view?usp=sharing